

Programmazione ad Oggetti - parte B: Esercitazione di laboratorio 7

Esercizio 1: Scrivere una classe Item da utilizzare come elemento di un `BinaryTree` che contenga un campo "nome" (stringa) e l'operatore "<". Creare un `BinaryTree` nel programma principale inserendo alcuni elementi e verificare il corretto attraversamento dell'albero secondo una delle modalità viste a lezione.

Esercizio 2: Scrivere una funzione membro pubblica "equal_tree_structure" della classe `BinaryTree` per determinare se un albero ha la stessa struttura di un albero il cui puntatore al nodo radice viene passato come argomento (due alberi A e B hanno la stessa struttura se sono entrambi vuoti, oppure se sono entrambi non vuoti e i sottoalberi destri e sinistri delle radici hanno rispettivamente la stessa struttura). E' consentito rendere "root" membro pubblico della classe `BinaryTree` per avere un accesso diretto.

```
int equal_tree_structure(Node<T>* theRootB)
```

Il metodo pubblico deve chiamare un metodo privato ricorsivo che accetta come parametro i puntatori ai due alberi

```
int equal_structure(Node<T>* theRootA, Node<T>* theRootB)
```

Esercizio 3: Scrivere una funzione membro pubblica "equal_tree" della classe `BinaryTree` per determinare se un albero è uguale ad un albero il cui puntatore viene passato come argomento (due alberi A e B sono uguali se hanno la stessa struttura e se tutte le coppie di nodi corrispondenti di A e di B hanno lo stesso valore).

E' consentito rendere "root" membro pubblico della classe `BinaryTree` per avere un accesso diretto.

```
int equal_tree(BinaryTree<T>* treeB)
```

Il metodo pubblico deve chiamare un metodo privato ricorsivo che accetta come parametro i puntatori ai due alberi

```
int equal(Node<T>* theRootA, Node<T>* theRootB)
```

Esercizio 4: Scrivere il distruttore della classe `BinaryTree`